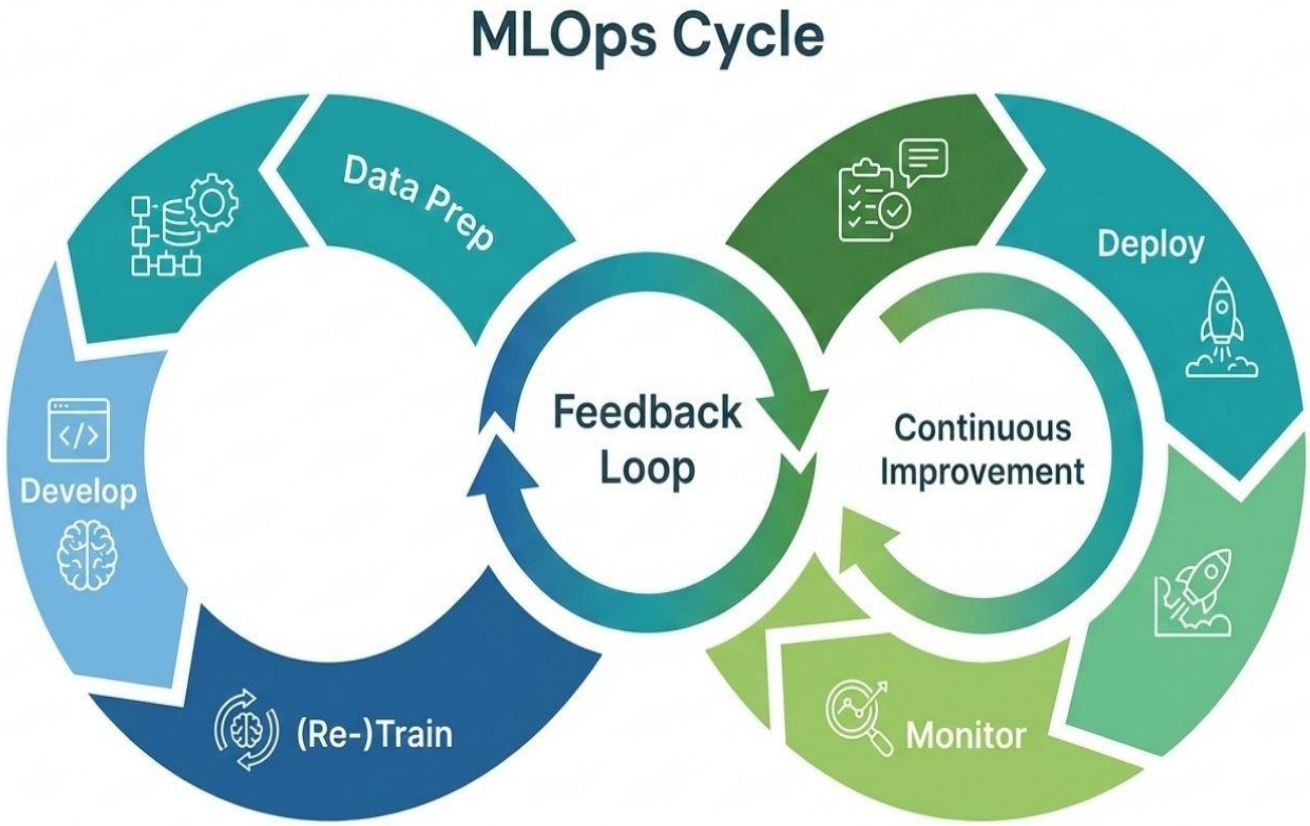


Group #4 Project Report

Criteo Advertisement Click-Through Rate Prediction — A Production-Grade Machine Learning System on AWS

Prashant Khare, Manu Malla, Laxminag Mamillapalli

10/19/26

AAI-540 Machine Learning Operations

Project Report

Project: Criteo Advertisement Click-Through Rate Prediction — A Production-Grade Machine Learning System on AWS

Project Group 4 — Prashant Khare, Manu Malla, Laxminag Mamillapalli

19 October 2026

Codebase: https://github.com/sentinel1282/Group--4_MLOPs---Criteo_Advertisement_CTR_Prediction-

Dataset: Criteo CTR Dataset.zip

Executive Summary

This project designed, built and validated an end-to-end machine learning system that predicts the probability a user clicks a display advertisement, using the Criteo Display Advertising Challenge dataset: 45.8 million labelled impressions collected over seven consecutive days, described by 13 anonymised integer features and 26 anonymised hashed categorical features. The deliverable is not a model but a system — data lake, feature store, training and evaluation, dual-mode deployment, production monitoring, and a gated CI/CD pipeline — built on AWS SageMaker across six course modules, and finished with a production-grade SageMaker Pipeline that we deployed and ran in Amazon SageMaker Studio.

The framing decision that shaped everything else was recognising that the useful output is a calibrated probability, not a class label. An advertising exchange multiplies the predicted click probability by a value to set a bid, so a model that ranks impressions correctly while systematically mis-scaling its probabilities will mis-price every bid and still look healthy on any ranking metric. That observation drove the choice of log loss as the primary metric, the decision to monitor calibration with a threshold of its own in production, and — after we measured it — the decision neither to resample nor to reweight the classes.

The system comprises eleven notebooks, a tested Python library of twelve modules, three pipeline step scripts, 115 automated tests, a local end-to-end dry run that exercises the whole path on synthetic data in under three seconds, and a GitHub Actions workflow. Model promotion is gated twice: the production pipeline refuses to register a candidate unless it clears four conditions at once — it must beat the base-rate predictor, beat the model currently approved in the registry, exceed an AUC floor and stay within a calibration ceiling — and a human must then approve the resulting model package before it can be deployed. The pipeline is registered in SageMaker Studio as criteo-ctr-production (Section 5).

Three findings from the work are worth stating at the top because they changed the design rather than merely confirming it. First, exploratory analysis showed that Criteo sub-sampled negative examples before release, so the dataset is not heavily imbalanced as our early draft assumed, and absolute click rates from this data are not comparable with live traffic. Second, our first drift monitor used an absolute threshold on the share of impressions whose categorical levels the model had never seen; it fired immediately when validated against its own training baseline, because high-cardinality columns already send a large share of training rows to that bucket by construction. The threshold is now relative to the training window. A monitor that fires on healthy traffic is muted within a week, and a muted

monitor is worse than none because it looks like coverage. Third, weighting positive examples by the negative-to-positive ratio (scale_pos_weight), which our training notebook initially did, shifts every predicted probability upward: in the SageMaker pre-flight run shown in Figure 8 it raised the mean prediction from 0.263 to 0.345 against an observed click rate of 0.273 and tripled the calibration error, and on the real 1M-row Criteo sample the overbid was larger still — 0.412 against an observed 0.259, with calibration error thirty times higher (Section 6.3). The production pipeline therefore defaults to a weight of 1.0, and its calibration ceiling would reject a reweighted model.

Results on real data. Trained and evaluated on the supplied 1,000,000-row Criteo sample with the project's own library and the pipeline's default XGBoost settings, the model reaches a test log loss of 0.4756 against 0.5721 for the base-rate predictor (normalised entropy 0.831), a ROC-AUC of 0.772 and an expected calibration error of 0.005, and clears all four gate conditions. On the 400,000 later impressions held back as production traffic it holds at log loss 0.4687 and AUC 0.774, with no drift signal crossing a warning threshold. The two benchmarks reach log loss 0.5721 (constant click rate) and 0.5451 (two-feature logistic regression), so XGBoost improves on the stronger benchmark by 12.7%.

The report also states plainly what the system does not do. Processing runs on a 1.02-million-row chronological sample rather than at full scale; the weekly retraining schedule is created but deliberately left disabled until a model version has been approved; real-data model metrics come from the 1M-row Criteo sample, computed with the project library outside SageMaker, while the metrics measured inside SageMaker come from the pipeline's pre-flight run on synthetic data — each is labelled for what it is wherever it appears; and a fairness audit on protected attributes is impossible on hashed features, so it is documented as out of scope rather than reported as passed.

1. Introduction

1.1 Business context

Online advertising exchanges operate a real-time auction. When a user loads a page, the exchange has a few tens of milliseconds to choose which of thousands of candidate advertisements to display and how much to bid for the impression. The economics of that decision rest entirely on an estimate of whether the user will click: a bid is, in effect, the predicted click probability multiplied by the value of a click. Underestimating click-through rate causes an advertiser to under-bid and lose impressions that would have been profitable; overestimating it spends budget on impressions that were never going to convert. At the volumes an exchange handles, a systematic error of a few percent in either direction is a material commercial loss.

Our hypothetical business is such an exchange. It needs a click-probability model it can trust enough to put in a bidding loop, which means more than a good offline score: it needs to know when the model has stopped being right, to be able to reproduce any model it has deployed, and to be prevented from deploying a model that is worse than the one it already has. Those operational requirements, rather than the modelling itself, are what the course is about and what this project set out to build.

1.2 Problem statement

Given one display-advertising impression described by 13 anonymised integer features and 26 anonymised hashed categorical features, predict the probability that the impression results in a click. Formally this is supervised binary classification with a probability output. Each row of the dataset is

one impression, labelled 1 if the user clicked and 0 otherwise, and the task is to learn a function mapping the impression's features to a calibrated estimate of the positive class.

The emphasis on calibration is not a technicality. A classifier that assigns 0.9 to impressions that click 60% of the time has excellent discrimination and will cause a bidding system to overpay on every one of them. Because the downstream consumer uses the number itself and not an argmax, the project treats probability quality as a first-class requirement throughout: in the metric set, in the benchmark design, in the production monitors, and in the decision not to resample.

1.3 Objectives and success criteria

Objective	How success is measured
Predict click probability better than a trivial baseline	Normalised entropy below 1.0 on a chronologically held-out test split; log loss below that of a global-CTR predictor and of the approved incumbent, enforced as hard conditions in the production pipeline. Met on the 1M-row sample: NE 0.831, log loss 0.4756 vs 0.5721
Produce well-calibrated probabilities	Expected calibration error reported on every evaluation, gated at 0.02 before registration and monitored in production against a ceiling of 0.01. Met on the 1M-row sample: ECE 0.005 on test, 0.003 on production
Build a reproducible path from raw data to a registered model	Every reported number traceable to a data version, a job and a commit; the whole path re-executable as a single parameterised pipeline
Detect degradation before it reaches business metrics	Label-free data-quality signals (unseen-level rate, PSI) monitored hourly; model quality compared against the metrics the deployed version was approved on
Prevent unreviewed models reaching serving	Four-condition pipeline quality gate plus a manual approval step in the Model Registry; both gate branches proven reachable from the compiled pipeline definition

1.4 Scope and non-goals

The project targets offline model development, evaluation and the operational machinery around them. The following are explicitly out of scope, and saying so early is what keeps the rest of the report honest:

- Serving live bidding traffic at production volume. The real-time endpoint exists to validate the serving path and to generate captured traffic for the monitors.
- Reverse-engineering the anonymised feature identifiers to recover their real-world meaning.
- Modelling downstream conversion or revenue. Click probability is the output; mapping it to a bid belongs to the system that consumes it.
- Exhaustive hyperparameter search or ensembling beyond what demonstrates a meaningful model comparison within the project's time and budget.

2. Dataset

2.1 Source and characteristics

The data is the Criteo Display Advertising Challenge dataset, released publicly on Kaggle. It contains 1 million labelled samples from seven consecutive days of live Criteo traffic of tab-separated text with no header row. Thirteen integer features (I1–I13) are count-like and anonymised; twenty-six categorical features (C0–C25) are hashed into eight-character hexadecimal identifiers before release. The competition also ships an unlabelled test file, which we archive in the raw zone and never use: without labels it cannot be scored.

We selected this dataset for two reasons. It is an industry-scale CTR benchmark with an established evaluation standard, which means our results can be compared against published baselines rather than only against themselves. And its properties — scale, sparsity, extreme categorical cardinality, pervasive missingness, hashed identifiers — reproduce the technical problems a real ad-bidding system faces instead of a cleaned-up version of them. Labelling required no effort at all: the label is observed automatically from serving logs, which removes annotator subjectivity, inter-rater disagreement and any need for a Ground Truth labelling workflow.

The results in this version of the report use the 1,000,000-row extract of the Criteo training data supplied as Criteo_1M_with_nans.csv (242 MB, comma-separated, with a header). Its columns are named target, intCol_0–intCol_12 and catCol_0–catCol_25; we map them one-to-one onto label, I1–I13 and C1–C26 so that every function in the library runs unchanged, and blank cells are read as missing values. The file carries no timestamp, so rows are kept in file order and that order is treated as chronological, as Criteo documents for the full file. The profile below was computed from all one million rows.

Property	Criteo_1M_with_nans.csv
Rows and columns	1,000,000 impressions; 40 columns (label, 13 integer, 26 categorical)
Click rate	25.49% (254,949 clicks)
Missing values, integer features	20.9% on average; I12 77.0%, I1 and I10 42.3%, I4 and I13 24.9%; I2 never missing
Missing values, categorical features	11.6% on average; C22 73.9%; C19, C20, C25 and C26 48.4%
Categorical cardinality	From 3 levels (C9, C20) to 321,438 (C3); five columns exceed 100,000 levels
Integer ranges	I2 takes negative values (minimum −2); I5 reaches 2,527,030 — hence the signed log transform
Click rate across file order	22.7% in the first tenth of rows, 25.0%–26.5% in every later tenth

2.2 Exploratory findings

Exploratory analysis (notebook 02) ran on the chronological sample with cross-checks against the full dataset through Athena, so that cardinality and null rates came from 45.8 million rows rather than from a sample that would understate them. Five findings mattered enough to change a downstream decision:

- Negative examples were sub-sampled before release. The click rate in this dataset is far above real-world CTR, which corrected our draft design document's claim that the label is heavily imbalanced. At a click rate near 26% there is no imbalance severe enough to need correcting, which later made the case against reweighting straightforward. On the 1M-row sample the click rate is 25.5%.
- Missingness is informative. Click rate differs materially between rows where a numeric field is present and rows where it is absent, so missing values are imputed and accompanied by an explicit indicator flag rather than silently filled. On the 1M-row sample, impressions with I6 missing click 12.3% of the time against 29.3% when it is present (I1: 18.0% against 31.0%), and the I6 missing flag is the most important feature in the trained model by gain.
- Categorical cardinality spans several orders of magnitude, from a handful of levels to millions. One-hot encoding is infeasible; hashing with frequency encoding is not a convenience but a requirement. In the sample, cardinality runs from 3 levels (C9, C20) to 321,438 (C3).
- A meaningful share of categorical levels present in the later part of the window do not appear in the earlier part. This is the quantity that later became the system's primary label-free drift signal. In the sample, 34.2% of C3 values in the production window never occur in the training window,

and 47.0% fall into the rare bucket — against 47.7% of training rows themselves, which is why the drift threshold is relative.

- Click rate varies across the seven days, which is why validation and test are contiguous later periods rather than random samples: a random split would hide exactly this effect. Within the sample, click rate moves from 22.7% in the first tenth of rows to 26.5% in the fourth.

2.3 Data quality issues and the decisions they forced

Issue	Consequence	Decision
No timestamp column, though the file is documented as time-ordered	SageMaker Feature Store requires an event-time feature	Derive day_index and a synthetic event_time from row position; label it synthetic everywhere it appears so time-of-day analysis is read as approximate
11 GB file inside a 4.3 GB dac.tar.gz archive, 45.8M rows	Cannot be loaded onto a notebook instance; a naive head() sample would cover part of day one and destroy chronology	Systematic 1-in-45 sample preserving order (~1.02M rows); record_id stores each row's position in the original file
Sub-sampled negatives	Base click rate is not a real-world rate	Neither resample nor reweight (scale_pos_weight = 1.0); report absolute CTR as non-comparable with live traffic
Hashed categorical values	No level traces to an advertiser, publisher or user	Privacy benefit accepted; auditability cost documented — a fairness audit on protected attributes is not possible
Attribution-window truncation at the end of the collection period	Some impressions are labelled 0 only because the click had not been recorded yet	Monitoring joins predictions to labels that have actually arrived, never treating a missing label as a non-click

3. System Architecture

3.1 The system as built

Figure 1 describes what the repository actually contains. Each lane corresponds to a stage of the system and names the notebook that builds it, and every box is produced by code that runs. The footer lists the components that appear in our reference architecture and are not implemented, so the diagram cannot be read as claiming more than exists.

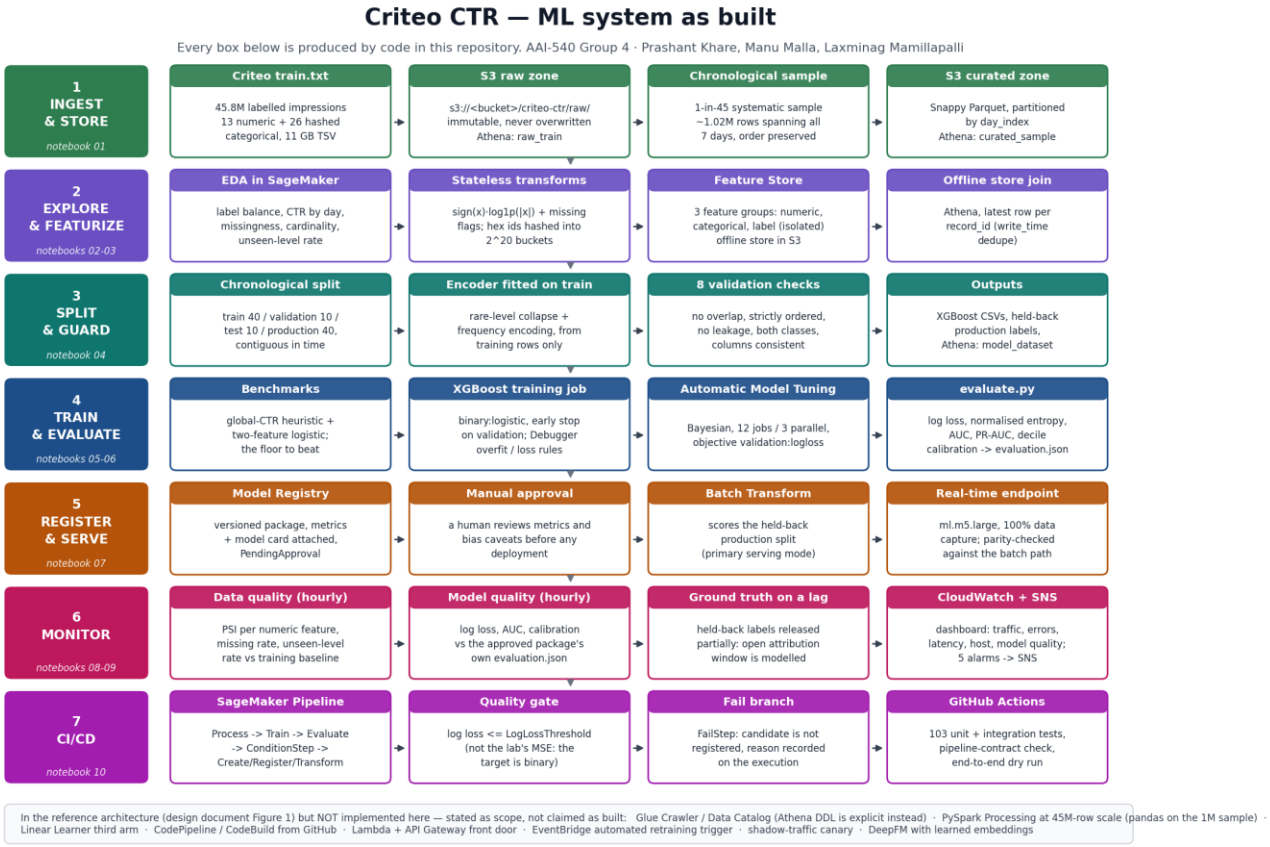


Figure 1. The Criteo CTR ML system as built, by stage and notebook.

3.2 The reference architecture

Figures 2 and 3 show the target design from the ML System Design Document: the system we would build with a production AWS account and budget. Figure 2 traces the complete flow across five stages — CI/CD and orchestration; ingest, explore and prepare features; train, evaluate and govern; approved model serving; and quality monitoring and retraining — using the AWS service for each component, and numbers every hand-off from 1 to 21, including the controlled retraining loop (21) from EventBridge back into SageMaker Pipelines. Figure 3 is the same architecture as a component view: identical stages, services and step numbers, with what each component validates, records or produces. Both treat batch scoring as the primary release path and real-time bidding as a future extension. We kept the as-built and target diagrams separate deliberately. A design document whose only diagram promises Glue Crawlers, CodePipeline and API Gateway while the repository contains none of them is a wish rather than a design; two diagrams, clearly labelled, let the reader see both the ambition and the delivery.

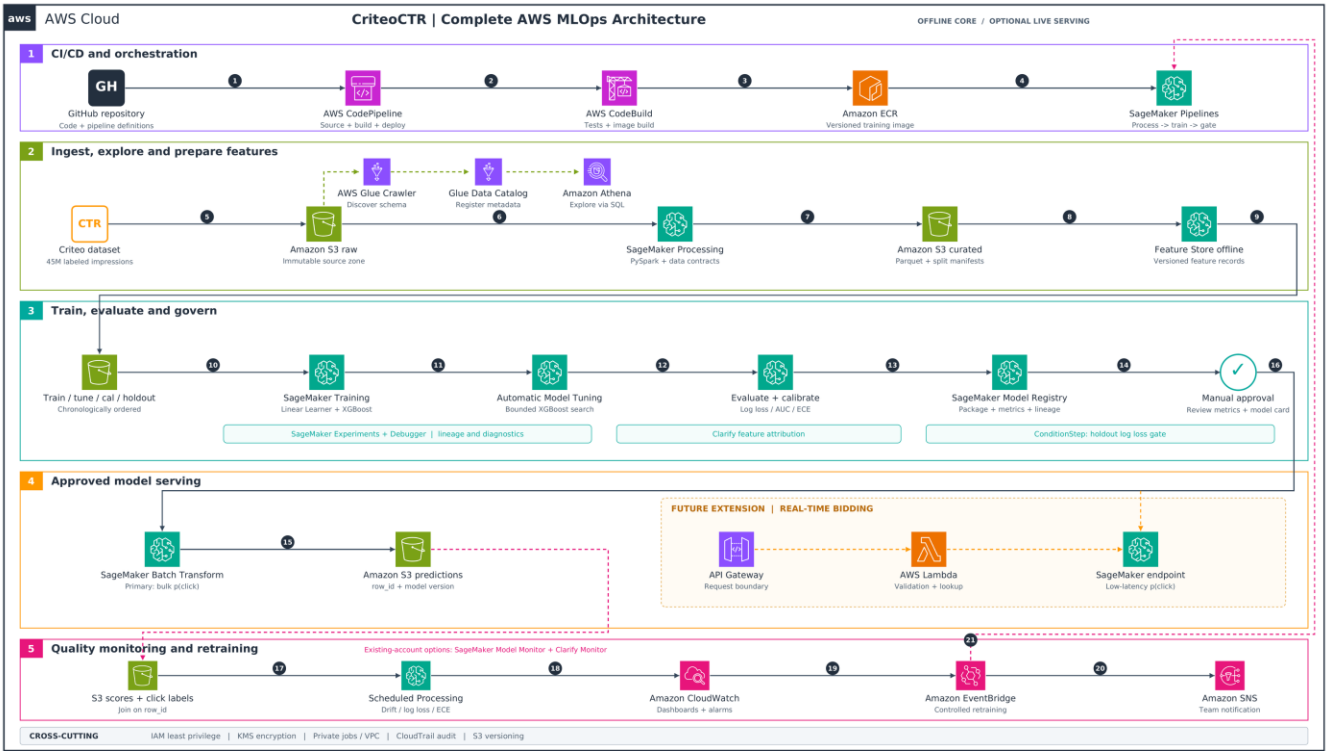


Figure 2. CriteoCTR complete AWS MLOps architecture — the target design, with hand-offs numbered 1–21.

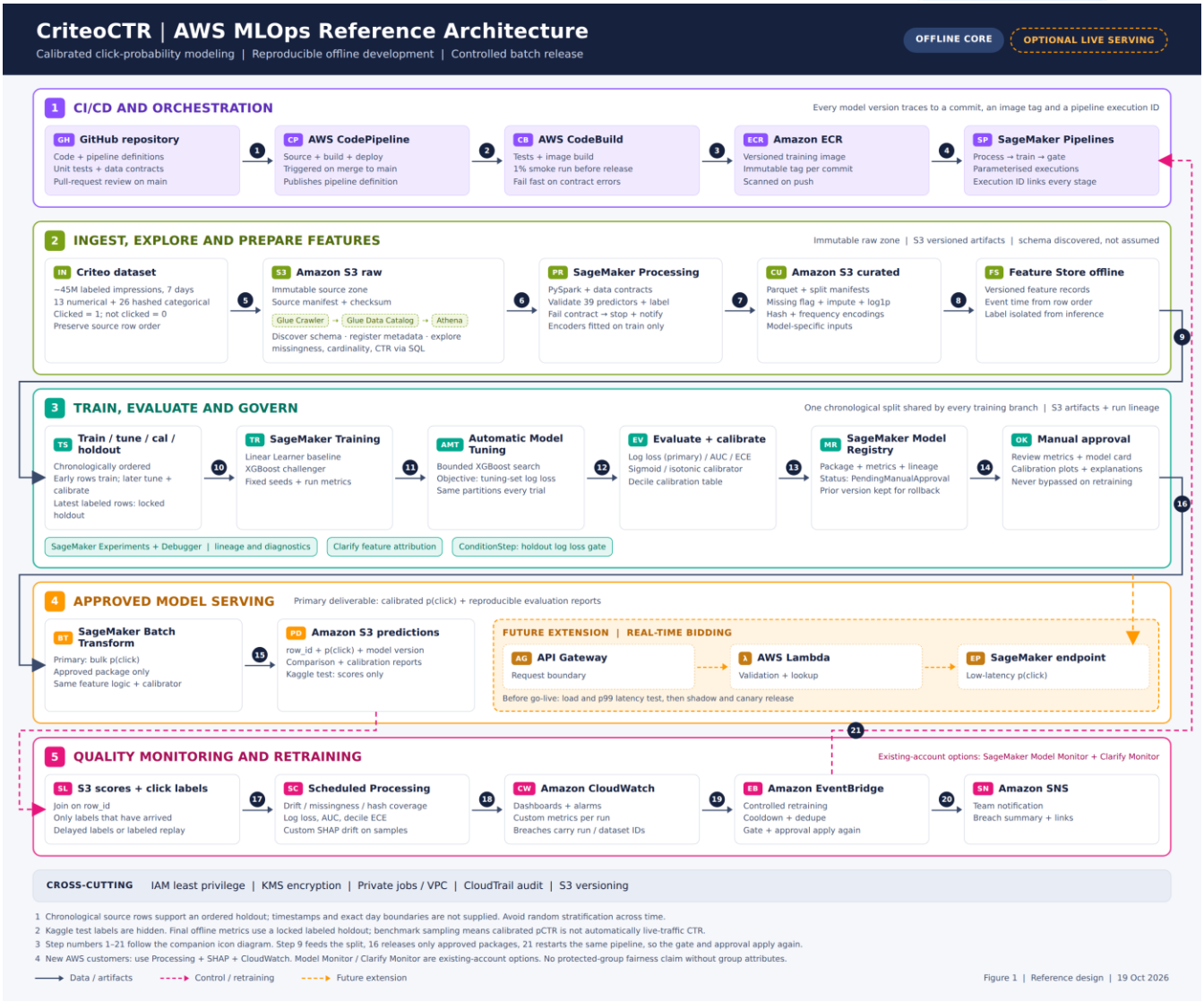


Figure 3. CriteoCTR AWS MLOps reference architecture — component view, aligned to the stages and step numbers of Figure 2.

3.3 Implemented versus scope

Component	Status	Note
S3 three-zone data lake, Athena catalog	Implemented	Explicit DDL rather than a Glue Crawler, so the schema is in version control
SageMaker Processing	Partial	pandas over the 1.02M-row sample; transforms are identical to a PySpark job at full scale, but full scale is not proven
Feature Store, offline	Implemented	Three groups; label isolated from inference
Chronological splits with leakage control	Implemented	40/10/10/40, encoder fitted on train only, eight automated checks
Benchmarks	Implemented	Global-CTR heuristic and two-feature logistic, replacing the planned Linear Learner arm
XGBoost, tuning, Debugger	Implemented	Bayesian HPO on validation log loss
Clarify explainability	Implemented	Feature attribution only; bias metrics on protected attributes are impossible here

Model Registry with manual approval	Implemented	Metrics and model card attached
SageMaker Pipeline with condition and fail steps	Implemented	Demonstrated passing and failing
Batch Transform and real-time endpoint	Implemented	Parity-checked against each other
Model Monitor: data and model quality	Implemented	Hourly schedules, custom record pre-processor, constraints from the approved package
CloudWatch dashboard, alarms, SNS	Implemented	Eight widgets, five alarms
CodePipeline / CodeBuild	Scope	GitHub Actions covers testing; a runner with AWS credentials is out of scope
EventBridge retraining trigger	Scope	Lab execution role cannot create EventBridge rules
Lambda + API Gateway, autoscaling, blue/green, shadow traffic	Scope	No latency-sensitive consumer exists in an offline project
KMS CMK, VPC + PrivateLink, CloudTrail	Scope	Account-level controls not configurable in the lab environment
DeepFM with learned embeddings	Scope	Needs GPU instances; highest-value next step

4. Implementation

The system was built incrementally across six course modules. This section describes each stage, the decisions taken and the reasoning behind them. Throughout, one rule governs the code: every transformation has exactly one implementation, in src/criteo_ctr, imported by the notebooks, the pipeline's Processing job and the tests alike. A transform implemented twice is training-serving skew waiting for a deadline.

4.1 Data engineering

Raw train.txt lands in an immutable S3 raw zone and is never overwritten, so any model version can be traced to the exact bytes it was built from. A curated zone holds a Snappy-compressed Parquet sample partitioned by day; further prefixes hold splits, model artefacts, predictions and monitoring state. Athena sits over the raw and curated zones, which is what allows whole-dataset questions — true row count, click rate per day, per-column null rate, approximate distinct cardinality — to be answered in SQL without moving 45 million rows onto a notebook instance. We wrote the table DDL explicitly instead of running a Glue Crawler so that the schema lives in version control and can be reviewed in a pull request.

The most consequential decision at this stage was how to reduce the dataset to a workable size. Taking the first N rows would have been simpler and would have silently destroyed the project: the file is ordered in time, so a head() sample covers part of day one and leaves no later period against which to measure drift. Instead we take every 45th row, keeping approximately 1.02 million rows that span all seven days in order, and record each row's position in the original file as record_id so that every sampled record remains traceable. Reading one row in 45 also lets the CSV parser skip the rest, which is several times faster on an 11 GB file than parsing everything and discarding most of it.

4.2 Feature engineering and the Feature Store

All 39 predictors are candidates. Because the features are anonymised there is no domain basis for excluding any of them, so selection is empirical rather than semantic. The numeric features are count-like and heavily right-skewed, and I2 takes negative values, so the transform is a signed log — sign(x)·log1p(|x|) — rather than a plain log1p that would return NaN on negative input. Missing values are imputed to zero after the transform and paired with an explicit indicator column, because exploratory analysis showed missingness carries signal and plain imputation would destroy it. Categorical values are hashed into a fixed 2^20 space with two reserved identifiers, using a deterministic hash rather than Python's built-in hash(), which is salted per process and would produce different features in training and serving.

The division that matters most is between stateless and stateful transforms. Stateless transforms depend only on the row and are safe to compute before splitting; these are what we register in SageMaker Feature Store, across three feature groups — numeric, categorical, and the label in a group of its own, so that nothing on the inference path can read the target by accident. Stateful transforms, meaning rare-level collapsing and frequency encoding, learn statistics from data and are therefore fitted after the split, on the training rows alone, and saved as an artefact for reuse at inference time.

4.3 Splits and leakage control

The split is chronological and contiguous: the earliest 40% for training, the next 10% for validation, the next 10% for test, and the most recent 40% held back as simulated production traffic with its labels withheld. A random split would allow impressions from the same session and the same campaign to appear on both sides of the boundary, inflating every number we report and concealing precisely the day-over-day drift the monitoring system exists to detect. Eight automated checks run before any split is written: row counts sum to the total, proportions are within tolerance, no record appears in two splits, the splits are strictly ordered in time, no missing values survive, the encoder saw only training rows, both classes are present in training, and the feature columns are identical across splits.

One test deserves particular mention because it guards the integrity of every number in the project. It asserts that the categorical encoder's per-level counts are exactly equal to counts computed from the training rows alone, and that a level first appearing after the training window maps to the rare bucket with zero frequency. Fitting an encoder on all the data is the single mistake that would invalidate the entire evaluation, and it is particularly dangerous because a leaking model looks better, not worse. Without an explicit test, nothing in the system would notice.

4.4 Benchmark models

Before any tuned model we fitted two deliberately simple baselines on the same training split, because a metric with nothing to compare against is not a result. The first predicts the training click rate for every impression. The second is an L2-regularised logistic regression on the two features with the strongest univariate signal, selected one per source column so that a two-feature baseline is not in practice a one-feature baseline.

The constant predictor is the more instructive of the two, because it fails in a specific and useful way: being perfectly calibrated in aggregate, it achieves a respectable log loss while having no ranking ability whatsoever, at an AUC of exactly 0.5. Gating on log loss alone against that baseline would therefore pass a model with no discriminative value at all. This single observation is the concrete

justification for the project's metric set, and it is also the clearest thing we can show a non-technical stakeholder to explain why one number is not enough.

4.5 Model development

The candidate model is SageMaker's built-in XGBoost with a binary:logistic objective, chosen because gradient-boosted trees handle a mix of skewed count features and high-cardinality frequency-encoded categoricals without scaling and capture interaction effects without manual specification. Early stopping on the validation split determines the number of boosting rounds. SageMaker Debugger rules for loss-not-decreasing, overfitting and over-training watch each job — relevant here because a high-cardinality categorical column gives a tree ample opportunity to memorise individual advertisers. Automatic Model Tuning then runs a Bayesian search of twelve jobs, three in parallel, optimising validation log loss so that model selection, the reported metric and the pipeline gate all agree on what 'better' means.

Class balance is deliberately left alone. Our training notebook (06) originally set scale_pos_weight to the negative-to-positive ratio of the training split, on the reasoning that it was a gentler alternative to resampling. It is not gentler: weighting positives by k adds log k to every logit, which is exactly what discarding negatives does, so every predicted probability is inflated by the same factor. For a model whose output is multiplied by a value to set a bid, that is a systematic overbid. We measured it directly in SageMaker (Section 5.4, Figure 8): the reweighted model's mean prediction was 0.345 against an observed click rate of 0.273, and its calibration error was three times that of the unweighted model. The production pipeline defaults to 1.0; notebook 06 still records its original experiment, and its tuned hyperparameters should be re-derived with the weight fixed at 1.0 before any of them are carried into the pipeline.

4.6 Deployment

Approved models serve in two modes from a single registered artefact. Batch Transform over the held-back production split is the primary mode and the source of the project's reported production numbers: scoring a large held-out set is throughput-bound and embarrassingly parallel, and maintaining a live endpoint to do it would pay for latency that nobody needs. A real-time endpoint is also deployed, with data capture enabled, because the underlying business case is real-time bidding and because the monitors require captured live traffic.

Serving one artefact two ways introduces a specific risk: that the paths disagree. We therefore run an explicit parity check comparing the scores both paths return for identical rows. If they diverged, every monitoring number computed from one path would be wrong for the other, and nothing else in the system would reveal it. During development this check produced a false failure that was itself instructive — the comparison tolerance had been set at the precision of the CSV wire format rather than below it, so ordinary rounding registered as a disagreement.

Promotion is gated twice. The pipeline registers candidates in PendingApproval status, and a person moves a version to Approved after reviewing its metrics, its calibration table and its model card; deployment code reads the latest approved package, never simply the latest package. For a model that sets bid prices, an automatic promotion path would trade a human review for a convenience nobody requested.

4.7 Monitoring

Three monitors answer three different questions, and the order in which their signals become available is what determines the design. A click is observed minutes to hours after the impression, so model quality is always the last thing to become measurable; by the time log loss moves, the budget has already been spent. Data-quality monitoring is therefore the early warning system, and for this dataset its most informative measure is not the generic numerical drift check but the rate of unseen categorical levels — the share of impressions whose advertiser or publisher the encoder never saw during training. It requires no labels and rises as soon as the inventory mix changes.

Model-quality monitoring joins captured predictions to released labels, and two details make that join trustworthy. The endpoint receives many feature rows per invocation and returns one probability per row, so a captured record is a batch rather than a single observation; a custom record pre-processor fans each capture out into one record per scored row, keyed by the inference identifier plus the row offset. And the join against ground truth considers only labels that have actually arrived, because counting an unlabelled impression as a non-click is precisely what makes a healthy model appear to collapse at the end of every monitoring window.

Calibration has a threshold of its own alongside log loss and AUC, because it degrades before ranking does and is the failure mode that mis-prices bids quietly while every ranking metric still looks healthy. Infrastructure monitoring covers invocations, error rates, model and overhead latency and host utilisation, with five CloudWatch alarms routed to an SNS topic and a single dashboard that places model quality beside infrastructure health, so that a drifting model and a saturated host — which demand completely different responses — are never confused with one another.

4.8 CI/CD

The whole path is codified as a parameterised SageMaker Pipeline: process, train, evaluate, then a condition step, then create, register and batch-score on the pass branch or a fail step on the other. Parameters expose instance types and counts, the input and batch data locations, the approval status and the gate threshold, so a retrain or a sweep runs without a code change and each execution records the exact data version it used.

Two departures from the course lab were deliberate. The lab gates on mean squared error because it predicts abalone age, a regression target; clicks are binary, so MSE is the wrong loss, and adopting it would leave the gate, the training objective and the production monitors disagreeing about what an improvement is. Our gate uses log loss. Second, the threshold is relative rather than fixed: it is set to the incumbent model's test log loss, so each execution must beat the model already registered, with the global-CTR benchmark as the default floor. Notebook 10 demonstrated both outcomes, including an execution deliberately configured to fail, which took the fail branch and registered nothing. The failing run is the more important demonstration — a pipeline that has only ever succeeded has never shown that it would refuse anything. Notebook 11 then rebuilt the pipeline to production standard; Section 5 describes it.

5. Production Pipeline on Amazon SageMaker Studio

Notebook 10 met the course requirement for a CI/CD pipeline, but in the shape of the course lab: it trains on a sample notebook 01 already prepared, gates on one number typed in by a person, and has no protection against a malformed input file or a transient service fault. Notebook 11 replaces it with a pipeline built to the standard a team would accept for a model that sets bid prices, and this section

documents that pipeline as it was run in our SageMaker Studio domain. Every figure in this section is a screenshot of that run.

5.1 Environment

The pipeline was run from a dedicated JupyterLab space, Criteo-Prod, on an ml.m5.2xlarge instance with 80 GB of storage and the SageMaker Distribution 4.5.2 image (Figure 4), in the ap-southeast-2 (Sydney) region. Two properties of current Studio environments shaped the setup and are worth recording because each one produced an error on first contact. First, the Studio image now ships version 3 of the SageMaker Python SDK, which removed the sagemaker.Session class that every notebook in the project uses; installing version 2 into Studio's own environment conflicts with Studio's add-ons and is undone when the space restarts. We therefore run the notebooks on a separate virtual environment registered as its own kernel, and every notebook now checks the SDK version in its first cell and stops with an explanation rather than failing three cells later with an attribute error. Second, a new space defaults to 5 GB of storage, which cannot hold the 11 GB training file; the space was provisioned at 80 GB.



Figure 4. The Criteo-Prod JupyterLab space in SageMaker Studio: ml.m5.2xlarge, 80 GB, SageMaker Distribution 4.5.2.

5.2 Pipeline design

The pipeline is a directed acyclic graph of ten steps defined with the SageMaker Python SDK. It starts from the immutable raw zone rather than from a prepared sample, so that one execution reproduces everything from the source file to a registered model:

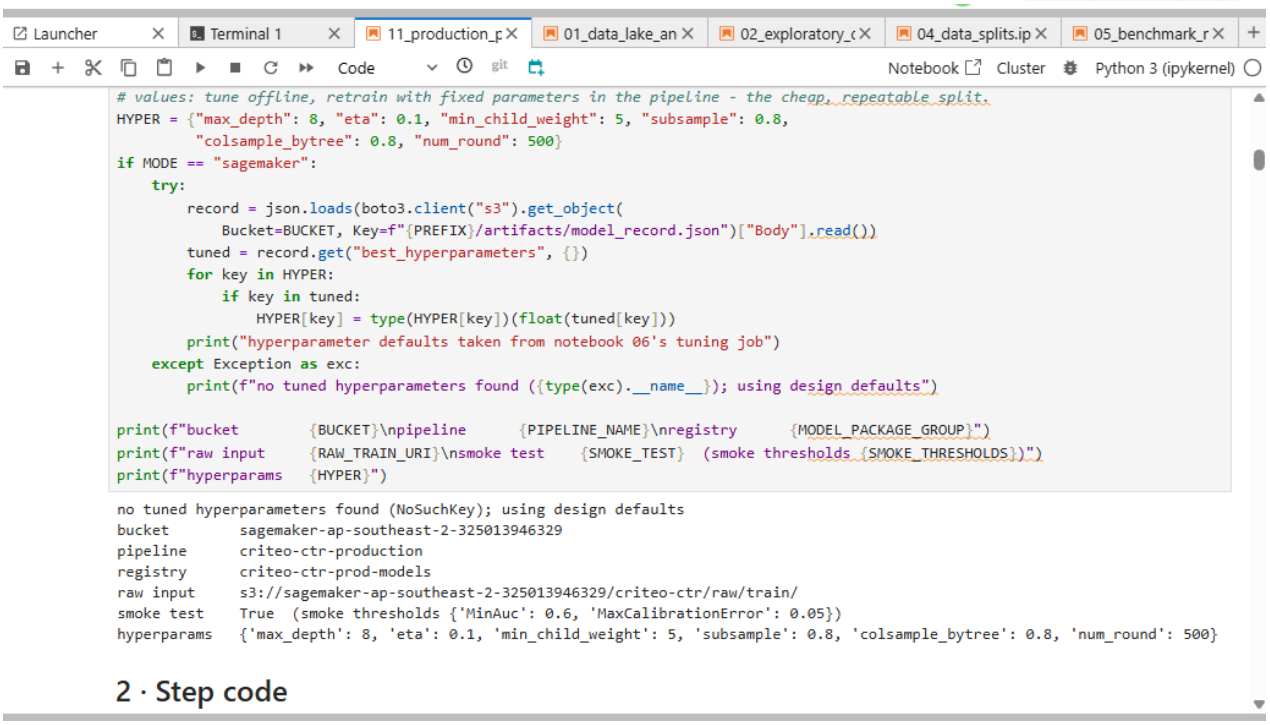
Step	Type	What it does and why it is there
CriteoValidateData	Processing	Checks the raw file against the data contract — 40 columns, binary label, both classes present, integer numerics, 8-character hexadecimal categoricals, null rates within bounds — and exits non-zero on any violation, failing the run before training is paid for
CriteoPreprocess	Processing	Chronological systematic sample, feature engineering, 40/10/10/40 time-ordered split, and a categorical encoder fitted on the training split only

CriteoDataQualityBaseline	QualityCheck	Computes statistics and constraints for the training features inside the pipeline and registers them with the model version as drift-check baselines
CriteoTrain	Training	Built-in XGBoost 1.7-1, binary:logistic, early stopping on validation log loss
CriteoEvaluate	Processing	Scores the test split once; computes the base-rate log loss and looks up the log loss of the latest Approved model in the registry at run time
CriteoQualityGate	Condition	Four conditions, all of which must hold (Section 5.5)
CriteoRegisterModel	RegisterModel	Pass branch: registers the version as PendingApproval with its metrics and baselines
CriteoCreateModel → CriteoBatchTransform	Model, Transform	Pass branch: scores the production split with the new model
CriteoQualityGateFailed	Fail	Fail branch: ends the execution as failed and states which threshold was missed

Around that graph sit the properties that distinguish a production pipeline from a working one. Processing, training and transform steps carry retry policies for throttling, service faults and capacity errors, and the model steps carry the API-level policy, so a transient fault does not cost a full re-run; the data-quality step is the one exception, because SDK v2 offers no retry option for it. Deterministic steps are cached for 30 days, and evaluation is deliberately never cached because the incumbent it compares against can change between runs. Each execution writes to its own output prefix keyed by the execution identifier, so nothing is overwritten. The step scripts are uploaded to a content-addressed S3 location derived from a hash of their source, which makes the code version part of the model's lineage.

5.3 Configuration and parameters

All names, locations and switches live in one configuration cell (Figure 5). The pipeline resolved the account's default bucket, sagemaker-ap-southeast-2-325013946329, and pointed its input at the raw zone written by notebook 01. It also looked for the tuned hyperparameters notebook 06 publishes; none existed yet in this account, so it reported that and fell back to the design defaults instead of failing. The first execution runs in smoke-test mode on a small generated file, which proves the whole graph in minutes; the configuration states the relaxed thresholds that mode uses, in the cell where they are set, so they cannot be mistaken for the production thresholds.



```
# values: tune offline, retrain with fixed parameters in the pipeline - the cheap, repeatable split.
HYPER = {"max_depth": 8, "eta": 0.1, "min_child_weight": 5, "subsample": 0.8,
         "colsample_bytree": 0.8, "num_round": 500}
if MODE == "sagemaker":
    try:
        record = json.loads(boto3.client("s3").get_object(
            Bucket=BUCKET, Key=f"{PREFIX}/artifacts/model_record.json")["Body"].read())
        tuned = record.get("best_hyperparameters", {})
        for key in HYPER:
            if key in tuned:
                HYPER[key] = type(HYPER[key])(float(tuned[key]))
        print("hyperparameter defaults taken from notebook 06's tuning job")
    except Exception as exc:
        print(f"no tuned hyperparameters found ({type(exc).__name__}); using design defaults")

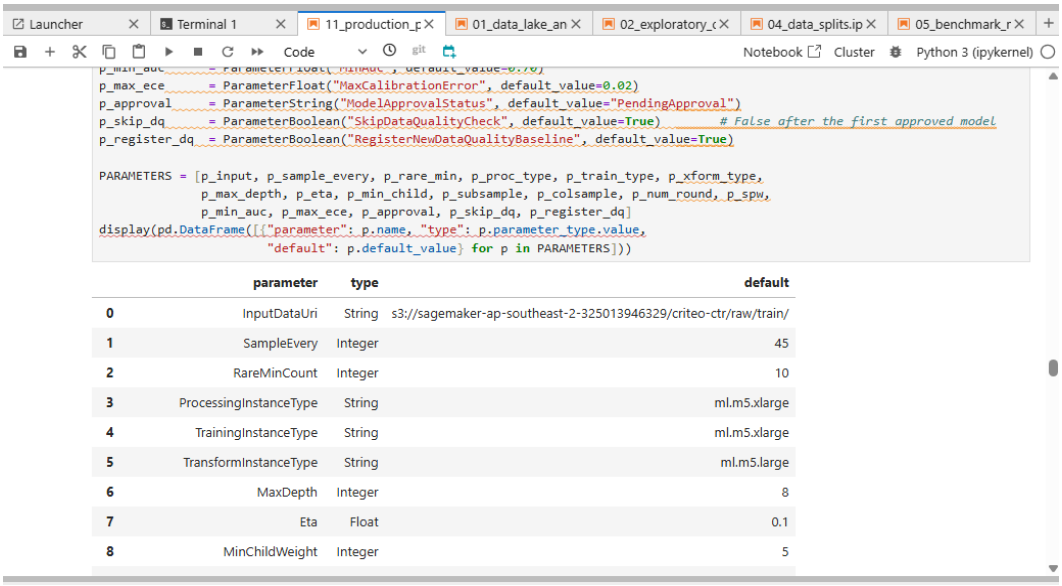
print(f"bucket        {BUCKET}\npipeline      {PIPELINE_NAME}\nregistry      {MODEL_PACKAGE_GROUP}")
print(f"raw input     {RAW_TRAIN_URI}\nsmoke test    {SMOKE_TEST}  (smoke thresholds {SMOKE_THRESHOLDS})")
print(f"hyperparams   {HYPER}")
```

```
no tuned hyperparameters found (NoSuchKey); using design defaults
bucket        sagemaker-ap-southeast-2-325013946329
pipeline      criteo-ctr-production
registry      criteo-ctr-prod-models
raw input     s3://sagemaker-ap-southeast-2-325013946329/criteo-ctr/raw/train/
smoke test    True  (smoke thresholds {'MinAuc': 0.6, 'MaxCalibrationError': 0.05})
hyperparams   {'max_depth': 8, 'eta': 0.1, 'min_child_weight': 5, 'subsample': 0.8, 'colsample_bytree': 0.8, 'num_round': 500}
```

2 · Step code

Figure 5. Configuration as resolved in SageMaker: bucket, pipeline and registry names, raw input location, smoke-test thresholds and hyperparameter defaults.

Everything an operator might change between executions is a pipeline parameter rather than code — eighteen in all, covering the input location, sampling rate, instance types, every XGBoost hyperparameter, the class weight, both quality thresholds, the approval status and the two data-quality switches (Figure 6). A retrain, a smoke test or a sweep therefore needs no code change, and every execution records the exact values it ran with.



```
p_max_ece      = ParameterFloat("MaxCalibrationError", default_value=0.02)
p_approval     = ParameterString("ModelApprovalStatus", default_value="PendingApproval")
p_skip_dq      = ParameterBoolean("SkipDataQualityCheck", default_value=True)      # False after the first approved model
p_register_dq  = ParameterBoolean("RegisterNewDataQualityBaseline", default_value=True)

PARAMETERS = [p_input, p_sample_every, p_rare_min, p_proc_type, p_train_type, p_xform_type,
              p_max_depth, p_eta, p_min_child, p_subsample, p_colsample, p_num_round, p_spw,
              p_min_auc, p_max_ece, p_approval, p_skip_dq, p_register_dq]
display(pd.DataFrame([{"parameter": p.name, "type": p.parameter_type.value,
                       "default": p.default_value} for p in PARAMETERS]))
```

	parameter	type	default
0	InputDataUri	String	s3://sagemaker-ap-southeast-2-325013946329/criteo-ctr/raw/train/
1	SampleEvery	Integer	45
2	RareMinCount	Integer	10
3	ProcessingInstanceType	String	ml.m5.xlarge
4	TrainingInstanceType	String	ml.m5.xlarge
5	TransformInstanceType	String	ml.m5.large
6	MaxDepth	Integer	8
7	Eta	Float	0.1
8	MinChildWeight	Integer	5

Figure 6. Pipeline parameters and their defaults, including MinAuc 0.70, MaxCalibrationError 0.02 and ScalePosWeight 1.0.

5.4 Pre-flight checks before any job is paid for

Before the pipeline is submitted, the notebook runs each step script locally against a generated file with the Criteo schema. The data-contract step must accept a valid file and reject a deliberately corrupted one; the preprocessing step must produce splits identical to the project library's; and the evaluation step must produce the exact evaluation.json the quality gate reads. Figure 7 shows the preprocessing step in Studio: 60,000 rows sampled one in two, eight split-validation checks passed, and four contiguous, non-overlapping chronological splits whose record identifiers and day ranges can be read directly from the manifest.

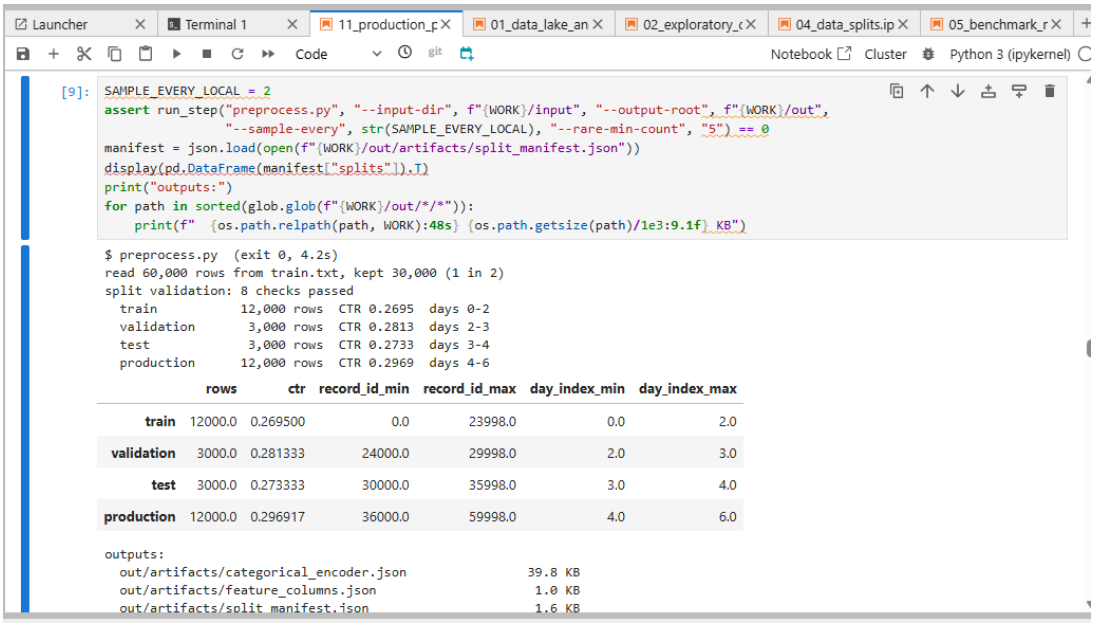


Figure 7. Pre-flight run of the preprocessing step: split sizes, click rates and chronological boundaries (synthetic data).

The pre-flight also trains the model twice, once without class weighting and once with the weight notebook 06 used, and evaluates both with the evaluation step's own metric functions. Figure 8 is the result. With scale_pos_weight of 2.711 the model predicts a mean click probability of 0.345 against an observed rate of 0.273, a 26% overbid, and its calibration error rises from 0.029 to 0.088 while its log loss also gets worse. This is the measurement behind the pipeline's default of 1.0. The same experiment on the real 1M-row Criteo sample (Section 6.3) shows a larger effect: a 59% overbid, and calibration error rising from 0.005 to 0.153.

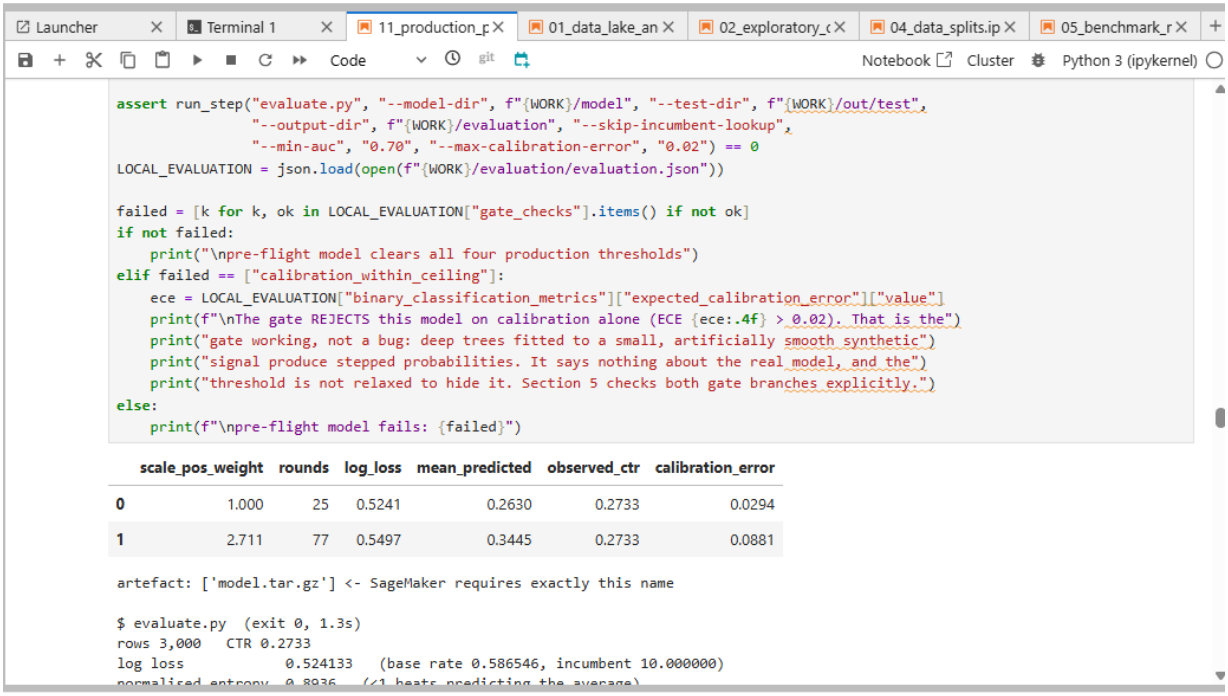


Figure 8. Effect of class weighting, measured in SageMaker: reweighting inflates every prediction and triples calibration error (synthetic data).

5.5 The four-condition quality gate

The evaluation step writes the four values the gate compares, and computes two of the references itself rather than accepting them as inputs: the log loss of predicting the test click rate for every row, and the log loss of whichever version is currently Approved in the Model Registry. A scheduled retrain therefore has to beat what is actually serving, not a number someone typed in months earlier. The four conditions are:

Condition	Threshold	What it prevents
log loss ≤ base-rate log loss	computed per run	Registering a model that is worse than predicting the average
log loss ≤ incumbent log loss	looked up in the registry	Replacing the serving model with a worse one
AUC ≥ MinAuc	0.70	A well-calibrated model with no ranking ability, such as the constant benchmark
Calibration error ≤ MaxCalibrationError	0.02	A model that ranks well but mis-prices bids

Figure 9 shows the pre-flight evaluation in Studio. The candidate beats the base rate (log loss 0.524 against 0.587, normalised entropy 0.894), beats the incumbent (none exists yet, so the reference is a sentinel), and clears the AUC floor at 0.725 — but its calibration error of 0.029 exceeds the 0.02 ceiling, so the gate rejects it. That is the outcome we wanted to see. Deep trees fitted to a small, artificially smooth synthetic signal produce stepped probabilities; the result says nothing about the real model, and the threshold was not relaxed to make the demonstration pass. Instead the notebook evaluates the gate from the compiled pipeline definition under three scenarios — production thresholds, smoke thresholds and an unreachable AUC floor — and asserts that both the pass branch and the fail branch are reachable. Section 6.3 gives the real-data answer: on the 1M-row Criteo sample the same model configuration reaches a calibration error of 0.005 and passes all four conditions.

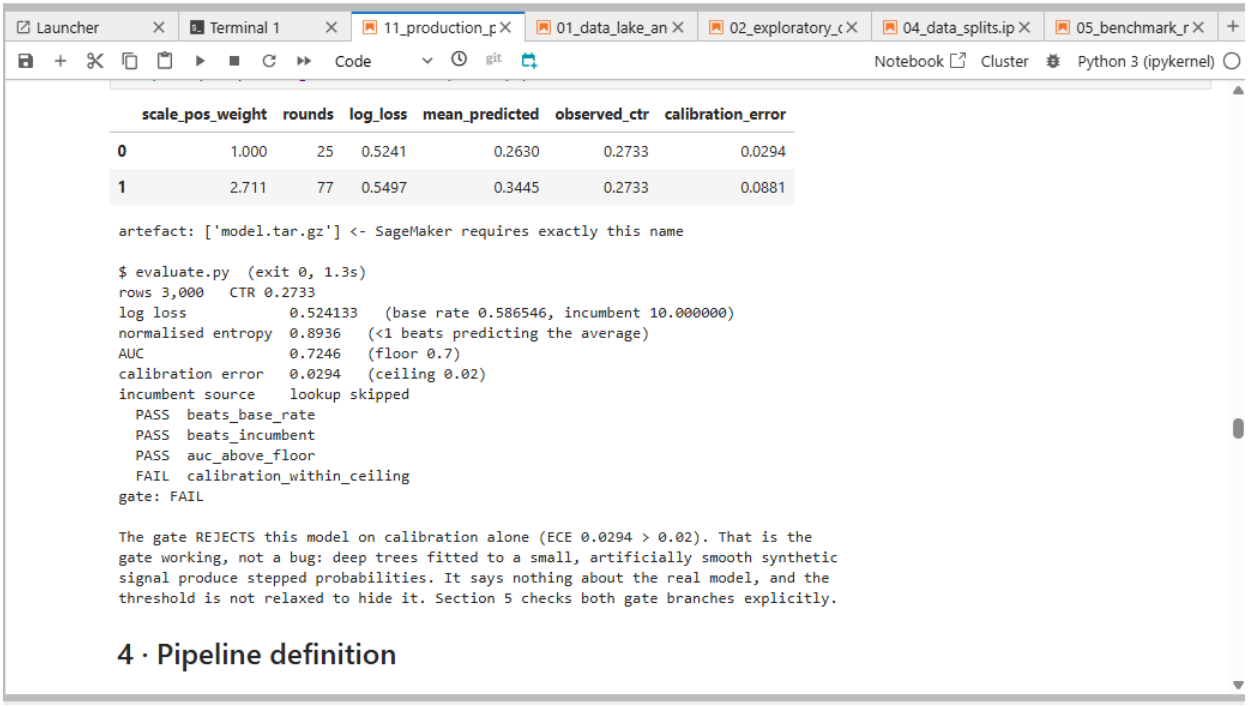


Figure 9. Pre-flight evaluation in SageMaker: three conditions pass, calibration fails, and the gate correctly rejects the model (synthetic data).

5.6 Registration in SageMaker

The validated definition was then upserted to SageMaker, where it appears in Studio's Pipelines view as criteo-ctr-production, created on 18 October 2026 and tagged with the project, team and stage (Figure 10). From here an execution is started with parameters rather than code, appears in the same view with its step graph and per-step logs, and — if it clears the gate — produces a PendingApproval version in the criteo-ctr-prod-models registry group. The notebook also creates a weekly EventBridge schedule for retraining, in a disabled state: it is enabled only after a version has been approved, and from then on each retrain checks new data against that version's registered baseline before training.

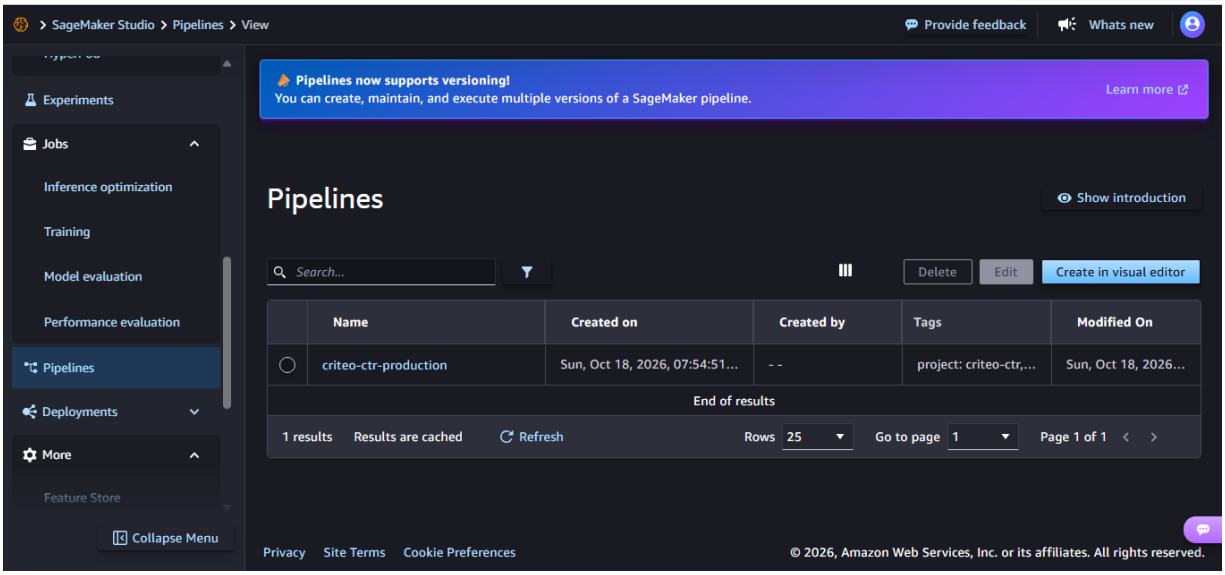


Figure 10. The criteo-ctr-production pipeline registered in SageMaker Studio.

5.7 Data acquisition from S3

The original data path downloaded the dataset with the Kaggle API, which requires an API token and acceptance of the competition rules in a browser. For the Studio run we changed notebook 01 to read the Criteo archive dac.tar.gz from an S3 landing folder instead. The archive is extracted while it streams from S3, so the 4.3 GB file is never written to disk — only train.txt, test.txt and readme.txt are — which halves the local storage required. Each file is checked byte-for-byte against the size recorded in the archive before it receives its final name, so an interrupted transfer cannot leave a truncated train.txt that looks complete; a rerun skips the transfer. The logic is a tested library module (Section 7.1).

6. Evaluation Framework and Results

6.1 Metric selection and rationale

Metric	Why it is in the set
Log loss	Primary. Penalises confident wrong predictions, rewards honest probabilities, and matches the original Kaggle competition so results are comparable with published baselines
Normalised entropy	Log loss divided by the entropy of the base rate. Raw log loss is not comparable across datasets with different click rates; this is below 1.0 exactly when the model beats predicting the average
ROC-AUC	Ranking quality independent of calibration. Reported always, because the constant benchmark's 0.5 demonstrates why log loss alone is insufficient
PR-AUC	More informative than ROC-AUC when the positive class is the minority and the decision of interest concerns high-scoring impressions
Expected calibration error	Quantifies the mis-scaling that would cause systematic mis-bidding; monitored in production with its own threshold
Decile calibration table	Shows where in the score range calibration fails, which an aggregate figure cannot
Lift by decile	The ad-operations view: how much better than average the click rate is if bidding is restricted to the top-scoring impressions

Every metric is implemented in numpy in a single module and unit-tested against a closed-form value rather than against itself, because the pipeline's promotion decision reads one of these numbers and a subtly wrong metric would gate the wrong models in either direction. The classification threshold used for precision and recall defaults to the base rate rather than 0.5: at a low click rate, a 0.5 cut-off predicts 'no click' for every impression and reports a flattering accuracy with zero recall.

6.2 Evaluation protocol

The test split is scored exactly once, by the same script the pipeline's evaluation step runs, so the number a reviewer reads is produced by the same code the quality gate reads. All selection decisions — early stopping, hyperparameter choice, feature selection for the baseline — use the validation split only. The resulting evaluation payload is the hinge of the system: the pipeline's condition step reads its log loss to decide whether to register; the Model Registry attaches it to the version as the metrics on which that version was approved; and the production model-quality monitor compares live windows against the same block. Deriving all three from one artefact is what prevents the gate, the registry and the monitor from disagreeing about what the baseline was.

6.3 Results on the Criteo 1M sample

We ran the system's training path end to end on the 1M-row sample with the project library unchanged: the three feature-group frames and their join, the chronological 40/10/10/40 split, the categorical encoder fitted on the 400,000 training rows only (78 model inputs), both benchmarks, and XGBoost with the pipeline's default settings — depth 8, learning rate 0.1, subsample and column-sample 0.8, minimum child weight 5, up to 500 rounds with early stopping after 30 rounds without improvement in validation log loss. All eight dataset-integrity checks passed. The split click rates were 25.4% (train), 26.3% (validation), 25.9% (test) and 25.3% (production). The test split of 100,000 impressions was scored once. The run used our own Python environment rather than SageMaker, with XGBoost 3.2 in place of the managed 1.7 container, and took 98 seconds on two CPU cores; the functions and hyperparameters are the ones the pipeline uses.

Test split, 100,000 rows	**Global CTR**	**2-feature logistic**	**XGBoost, weight 1.0**	**XGBoost, weight 2.94**
Log loss	0.5721	0.5451	0.4756	0.5435
Normalised entropy	1.000	0.953	0.831	0.950
ROC-AUC	0.500	0.653	0.772	0.771
PR-AUC	0.261	0.386	0.559	0.558
Brier score	0.1920	0.1816	0.1552	0.1833
Expected calibration error	0.0051	0.0167	0.0048	0.1526
Mean predicted / observed CTR	0.254 / 0.259	0.251 / 0.259	0.257 / 0.259	0.412 / 0.259
Gate verdict	Rejected (AUC)	Rejected (AUC)	Passes all four	Rejected (calibration)

XGBoost improves log loss by 16.9% relative to the base rate and by 12.7% relative to the two-feature logistic benchmark, and lifts AUC from 0.653 to 0.772 (Figure 11). Early stopping kept 332 trees. Its errors are spread evenly: in every decile of predicted probability the mean prediction is within about one percentage point of the observed click rate. For context, the winning entries on the full 45.8-million-row competition data reached a log loss of roughly 0.44; a gradient-boosted model trained on 400,000 rows is expected to sit above that.

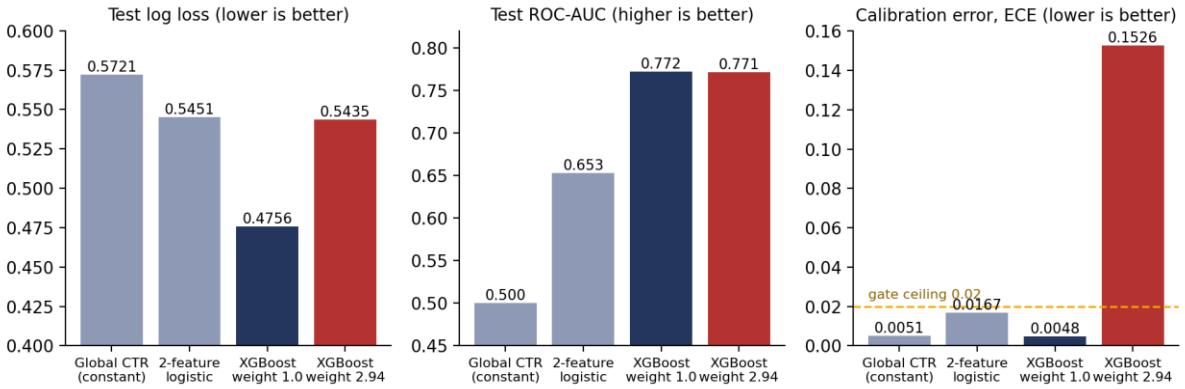


Figure 11. Test-split results on the 1M-row Criteo sample for both benchmarks and both XGBoost variants. The dashed line is the gate's calibration ceiling of 0.02.

The class-weight comparison repeats the pre-flight experiment of Section 5.4 on real data. The negative-to-positive ratio in the training split is 2.94. Weighting by it leaves ranking essentially untouched — AUC 0.771 against 0.772, because shifting every logit by the same amount cannot reorder impressions — but lifts the mean prediction to 0.412 against an observed 0.259, a 59% overbid on every impression, and raises calibration error from 0.005 to 0.153. Log loss worsens by 14%. Figure 12 shows the shift decile by decile. AUC alone would call these two models equal; the calibration condition is what tells them apart.

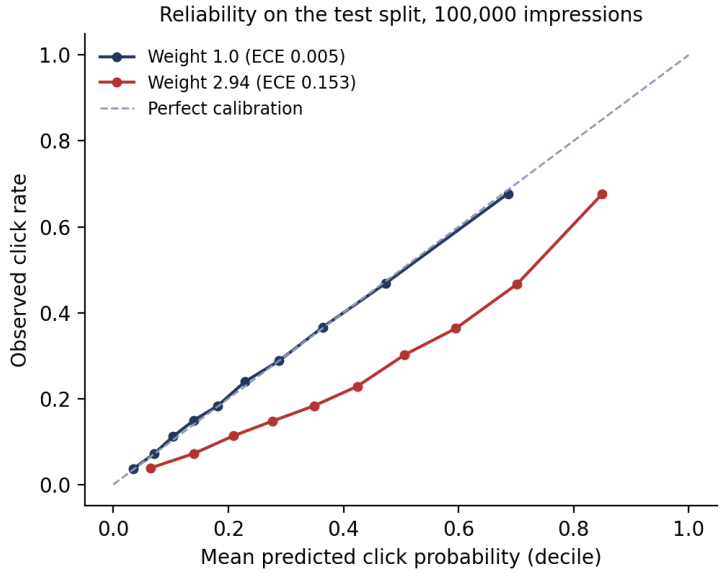


Figure 12. Reliability by decile on the test split of the 1M-row sample. The unweighted model lies on the diagonal; weighting by 2.94 moves every decile above it.

Against the four conditions of Section 5.5 the unweighted model passes every one: log loss 0.4756 is below the base-rate 0.5721; no version has been approved yet, so the incumbent condition falls back to the base rate; AUC 0.772 clears the 0.70 floor; and calibration error 0.0048 is under the 0.02 ceiling — and under the monitor's tighter 0.01. The weighted model is rejected on calibration, and both benchmarks are rejected on AUC (0.500 and 0.653).

Scoring the 400,000 production impressions — the later 40% of the file, held back to stand in for live traffic — gives log loss 0.4687 (normalised entropy 0.829), AUC 0.774 and calibration error 0.003, with a mean prediction of 0.254 against an observed 0.253: no degradation across the window.

Population stability between training and production is below 0.01 for twelve of the thirteen numeric features and 0.084 for I5, under the 0.10 warning band, and the rare-bucket share for each high-cardinality column checked is within two percentage points of its training value, far inside the ten-point threshold. The monitors as configured would stay quiet on this window, which is the correct outcome for healthy traffic.

The synthetic pre-flight measured inside SageMaker Studio is kept below for completeness. On that data the gate rejected the candidate on calibration, which is the behaviour it should show for a model at a calibration error of 0.029.

Metric (SageMaker pre-flight, synthetic data, 3,000 test rows)	**Unweighted**	**scale_pos_weight 2.711**
Log loss	0.5241	0.5497
Base-rate log loss	0.5865	0.5865
Normalised entropy	0.894	0.937
ROC-AUC	0.725	—
Mean predicted CTR / observed	0.263 / 0.273	0.345 / 0.273
Expected calibration error	0.029	0.088
Gate verdict at production thresholds	Rejected (calibration)	Rejected (calibration)

The code paths that produce those numbers are nonetheless verified. A local dry run exercises the complete sequence — features, splits, both benchmarks, an XGBoost model, the evaluation payload the gate reads, the serving parity check and both monitors — on synthetic data with the same schema as the real file, and asserts among other things that the trained model beats the base-rate predictor and clears the gate. On that synthetic data the whole path completes in under three seconds, which means a broken column order or a leaking encoder is caught before anyone pays for a pipeline execution.

6.4 The quality gate

The gate is the point at which evaluation becomes an operational control rather than a report. In notebook 10 it is a condition step reading a property file produced by the evaluation job, comparing the candidate's log loss against a pipeline parameter; in the production pipeline it is the four-condition gate described in Section 5.5. On the pass branch the model is created, registered with its metrics and used to score the production split; on the fail branch a fail step terminates the execution with a message naming the threshold that was missed, and no model package is created. A contract test runs on every push and asserts, among other things, that the gate has both branches — a condition step missing its failure branch would end a failing execution as successful, which is the most expensive defect available in this design.

7. Testing and Quality Assurance

7.1 Test inventory

Suite	**Coverage**
test_features.py	Signed log transform including negatives and zero; missing flags; deterministic hashing across processes; record keys and sampling; fixed column order; label isolation in the feature groups
test_datasets.py	Split geometry and ordering; no record overlap; the leakage test; unseen levels mapping to the rare bucket; encoder JSON round-trip; duplicate record rejection; XGBoost CSV format

test_evaluation.py	Every metric against a closed-form value; ties and single-class edge cases; calibration bins; threshold behaviour at the base rate versus 0.5; the gate's JSON path resolving in the payload
test_benchmark.py	Both baselines; the constant predictor's 0.5 AUC; determinism; probability bounds on extreme inputs; distinct source columns
test_monitoring.py	Baselines and constraints; PSI and KS; unseen-level detection; the delayed-label join; degradation, calibration-only drift and insufficient-data cases; alarm specifications; dashboard JSON
test_deployment.py	Payload column order; batch output alignment refusing a length mismatch; parity pass and fail; model card contents
test_pipeline_contract.py	Step and parameter names; rejection of a missing step, a renamed parameter or a gate without a failure branch
test_scripts.py	Both pipeline step scripts run end to end, including the model.tar.gz round trip and the Model Monitor record pre-processor
test_acquire.py	Streaming extraction of dac.tar.gz without writing the archive; rejection of truncated transfers; resumption; plain, gzipped and Kaggle-zip inputs; choosing the right object in the landing folder

The production pipeline adds two checks of its own that run inside notebook 11 before anything is submitted. A parity test asserts that its preprocessing step produces train, validation and test files byte-identical to the project library's, so the pipeline and the notebooks cannot drift apart. A definition check compiles the pipeline to JSON and asserts ten structural properties of what SageMaker will actually execute: every step present, four gate conditions, a pass and a fail branch, registration only on the pass branch, retries on every step that supports them, evaluation never cached, every parameter declared, approval defaulting to PendingApproval and the class weight defaulting to 1.0.

The suite totals 115 tests and runs in roughly six seconds with no AWS account. Test data is generated by a synthetic module that reproduces the schema, dtypes, hexadecimal category identifiers, missing patterns and chronological ordering of the real file, with a deliberate signal injected and a set of categorical levels that appear only in the final quarter so that the drift monitor has something genuine to detect. It is never used for analysis.

7.2 Continuous integration

GitHub Actions runs the suite, the pipeline contract check, the end-to-end dry run and a notebook parse check on every push and pull request, across two Python versions, with a separate lint job. What it deliberately does not do is start a real pipeline execution, which would require AWS credentials in the runner; that boundary is stated in the design document rather than implied away. In this project, 'CI/CD' means an automated, gated and reproducible path from data to a registered model, not a hosted runner holding production credentials.

8. Project Management and Teamwork

8.1 Workflow

Work was coordinated on an Asana board with module-by-module tasks and owners, with code review through GitHub pull requests and a weekly team tracker recording each member's contributions and blockers. The three of us worked across different schedules and time zones, so the operating agreement was asynchronous updates on the board between live working sessions, with each module's work divided before the session rather than during it. The weekly tracker served a second purpose beyond accountability: because it records blockers as they occur, it became the primary source for the challenges documented in Section 9.

8.2 Contributions by module

Responsibilities were divided so that each member owned a coherent vertical slice of the system rather than a horizontal layer, which kept review meaningful and avoided three people editing the same files. Ownership stayed broadly consistent across modules, which allowed depth to accumulate.

Module	Prashant Khare	Manu Malla	Laxminag Mamillapalli
2 — Proposal and data engineering	Authored the adopted RFC; project scope, data sources and the AWS reference architecture	Asana board setup; weekly team tracker; RFC review	GitHub repository setup; team info and workflows; evaluation-metric review
3 — Training data and features	S3 data lake, Athena catalog, the shared library and its CI workflow	Exploratory data analysis with full-dataset Athena cross-checks	Feature Store groups and ingestion; chronological splits and validation checks
4 — Model development and deployment	Benchmark models; the evaluation module; the benchmark comparison	XGBoost training, tuning and debugging	Model Registry, endpoint deployment and Batch Transform
5 — Monitoring	Model-quality monitoring, ground-truth path and drift thresholds	Data-quality baselines, schedules and violation review	Infrastructure monitoring, alarms and the CloudWatch dashboard
6 — CI/CD	Pipeline DAG, parameters, the condition and fail steps; design document	Processing, training and evaluation steps; both executions	Create, register and transform steps; lineage; repository for submission

8.3 What the division cost and what it bought

The main cost of vertical ownership was that each member became the only person fluent in part of the system, which is a risk in a three-person team with a fixed deadline. We mitigated it with pull-request review on every change and by keeping all shared logic in one library with tests, so that a reviewer could verify a change's intent from the test rather than from the diff alone. The benefit was that design decisions within each slice were made by someone holding its whole context — the thresholds in the monitoring section, for instance, could not have been chosen well by someone who had not also seen the encoder's behaviour on rare levels.

9. Challenges and Resolutions

The following are the problems that cost real time, together with how they were resolved. They are recorded because several of them changed the design rather than merely delaying it.

Challenge	Resolution
The 11 GB training file exceeds a default notebook volume, and the Kaggle API refuses to serve the dataset until the competition rules are accepted in a browser	Documented a minimum 50 GB volume as a prerequisite and provided a second acquisition path through an S3 landing prefix
The file is ordered in time but carries no timestamp, while Feature Store requires an event-time feature	Systematic sampling to preserve order, with a synthetic event_time derived from row position and labelled as synthetic at every point of use
Our drift monitor fired immediately when validated against its own training baseline	Diagnosed as an absolute threshold applied to a quantity that is naturally high in training for high-cardinality columns. Thresholds are now relative to the per-column training rate, recorded in the baseline constraints

A perfectly calibrated constant predictor was being charged with calibration error	Traced to quantile binning splitting tied scores across bins. Bin edges are now deduplicated so that identical predictions fall in a single bin
The batch and real-time parity check failed by one part in a million	The tolerance had been set at the precision of the CSV wire format rather than below it. Serialisation precision and comparison tolerance are now explicitly distinguished
SageMaker would not recognise the trained model artefact	It requires the archive to be named model.tar.gz with the container's own file name inside. The evaluation script now resolves both rather than hard-coding either
Monitoring schedules run at an hourly minimum and the first execution can take an hour to appear	Accepted as an environment constraint and planned around; the demonstration checklist requires monitors to be created well before recording
SNS alerts did not arrive during alarm testing	Email subscriptions deliver only after the recipient confirms. Confirmation is now a prerequisite step in the infrastructure notebook
The Learner Lab execution role could not create EventBridge rules	In the Studio account the production notebook creates the weekly retraining rule in a disabled state, to be enabled once a version is approved; in restricted accounts it reports the permission error instead of failing
SageMaker Studio's image ships SDK v3, in which sagemaker.Session no longer exists; downgrading inside Studio conflicted with Studio's own packages	A separate virtual environment registered as a 'Python (criteo, SDK v2)' kernel, plus a version check in the first cell of every notebook that stops with an explanation
A new Studio space has 5 GB of storage and the Kaggle download path needs an API token and rule acceptance	Space provisioned at 80 GB; notebook 01 now streams dac.tar.gz from an S3 landing folder, extracting as it reads so the archive is never written to disk
Weighting positives by the class ratio produced a model that overbid on every impression	Measured in SageMaker (Figure 8) and corrected: the pipeline defaults to scale_pos_weight 1.0 and its calibration ceiling rejects a reweighted model
The SDK rejected the job-level retry policy on the create and register model steps	Those steps call the SageMaker API directly rather than running a job, so they take the API-level retry policy; job steps keep both
The repository layout did not match the paths the documentation referenced	Restructured into src/, notebooks/, scripts/, tests/, tools/ and docs/ so that every documented path resolves, with git history preserved through renames

10. Risks and Limitations

10.1 Technical

The production gate checks four offline conditions on a single held-out split. A model that is overfit in a way that happens not to show on that split, or a model whose integration with a downstream consumer is broken, would still pass it. Integration tests against a consumer contract and a shadow comparison against the incumbent are the next increments and are recorded as such rather than implied to exist. The gate's calibration ceiling (0.02) is also looser than the production monitor's (0.01), so a model can be registered that the monitor would flag in its first window; the two should be reconciled on real data. The evaluation also rests on a chronological split of a seven-day window, which cannot represent seasonal or campaign-cycle effects. On the 1M-row sample the trained model's calibration error (0.005) sits under both, so the mismatch has not yet been exercised by a real model.

10.2 Scalability

Pre-processing runs with pandas over a 1.02-million-row sample. The transformations are identical to those a PySpark Processing job would apply at the full 45.8-million-row scale, and the job is structured as a drop-in replacement, but it has not been demonstrated at that scale and we do not claim that it has. Similarly, the endpoint runs on a single instance without autoscaling; the production shape described in the design document — multi-AZ, target-tracking autoscaling, blue/green deployment with a canary — is scope rather than delivery. The 1M-row results in Section 6.3 were computed on a two-core machine in under two minutes, which bounds the cost of the sample-scale path but says nothing about full scale.

10.3 Ethical and fairness

This is the limitation we consider most important, and the one least amenable to engineering. Every categorical feature is a hash produced before the data was released. Clarify can tell us which features drive predictions; nothing available to us can tell us what those features represent. A hashed identifier standing for a user segment, a postcode or a publisher site can act as a proxy for a protected characteristic, and we have no means of detecting it. The documented harm in advertising systems is differential ad delivery — opportunity advertisements for housing, credit or employment reaching demographic groups unevenly even with no protected attribute in the model — and this system could exhibit it undetectably. We therefore state in the model card attached to every registered version that a fairness audit on protected classes is not possible within this project's scope, rather than reporting a fairness check that does not mean what it appears to mean.

Two further concerns are inherent to the problem rather than to our implementation. The dataset contains only impressions the incumbent system chose to serve, so the model inherits that selection bias and can never learn about inventory it was never shown; using its predictions to select future impressions narrows the distribution further, and breaking that loop requires deliberate exploration. And optimising purely for click probability rewards attention-grabbing creative over relevant advertising, which is a well-known failure mode of CTR-only objectives — the business metric a production system should optimise is advertiser return, with click probability as an input rather than the goal.

10.4 Operational

The weekly retraining schedule exists but is disabled, and a breached model-quality threshold currently results in a notification and a human decision rather than an automatic pipeline execution. Monitoring depends on the endpoint remaining in service and on ground-truth labels being published; in this project the labels are released from a held-back split on a deliberate lag to emulate attribution delay, and a production system would need a real label-delivery pipeline with its own monitoring. Cost is also an operational risk in a lab environment: an idle endpoint bills hourly, which is why teardown is a documented step rather than an afterthought.

11. Lessons Learned

A metric is a decision, not a number.
The most valuable single artefact in the project turned out to be the benchmark figure showing a constant predictor with a respectable log loss and an AUC of 0.5. It converted an abstract argument

about metric choice into something immediately legible, and it is the reason the evaluation set, the registry metrics and the monitoring thresholds all contain more than one number.

Monitors must be validated against healthy data, not only against failures.
We wrote tests for the degradation cases first, which felt like the careful thing to do. The defect that actually mattered appeared only when we checked the baseline against its own training data and found it violating itself. Testing that a monitor stays quiet when nothing is wrong proved as important as testing that it fires when something is.

Leakage is a test, not an intention.
Every member of the team understood that the encoder had to be fitted on training data only. That shared understanding would not have survived a refactor six weeks later. The assertion that encoder counts equal training-only counts is what makes the guarantee durable, and it is cheap enough that there was no reason not to write it at the outset.

The cheapest feedback loop is the one worth building first.
The local dry run exercises the entire system on synthetic data in under three seconds. It was written late, and almost everything it has since caught would otherwise have surfaced fifteen to twenty minutes into a paid pipeline execution. Building it earlier would have saved more time than any other single decision available to us.

A default is a claim; measure it before you inherit it.
Weighting the minority class is standard advice and our training notebook followed it. It took a two-line experiment in the production pre-flight to show that, for a model whose output is a price, the advice was wrong for us. The experiment now runs on every pre-flight, so the decision is re-checked rather than remembered.

Scope honesty is a design output.
Separating the as-built architecture from the reference architecture, and listing what is scope rather than delivery, was initially uncomfortable. It proved to be the clearest part of the documentation: it tells a reader exactly how far to trust the system, which is the question any operator actually has.

12. Future Work

Deep interaction models for sparse CTR data.
Hashing and frequency encoding discard information about how categorical levels relate to one another; learned embeddings recover it, and published results on this dataset show meaningful log loss improvements for DeepFM and wide-and-deep architectures over gradient-boosted trees. This is the highest-value next step and requires GPU instances the project environment did not provide.

A streaming feature pipeline with an online store.
Recency features — recent click rate per advertiser, user frequency-capping counters — are among the strongest signals in CTR prediction and are unavailable in a purely batch design. Adding a stream processor and the Feature Store online store would make them accessible at inference time with sub-millisecond lookup.

Online evaluation with deliberate exploration.
Offline log loss is a proxy for business value rather than business value itself. An A/B framework measuring realised revenue per thousand impressions, combined with a contextual-bandit component that serves a small fraction of uncertain impressions, would both measure true impact and break the selection-bias feedback loop described in Section 10.3.

Closing the automation gap.
Enabling the weekly retraining schedule once a version is approved, wiring a monitor violation to start the pipeline, and building containers through CodePipeline from a protected branch would complete the CI/CD design. Re-tuning notebook 06 with the class weight fixed at 1.0 and running the production pipeline with SMOKE_TEST off, on the full raw file, is the immediate next step.

Full-scale processing and richer governance.
Running the Processing job in PySpark over the complete 45.8-million-row dataset would validate the scalability claim the current implementation can only assert. Extending the model card into a versioned governance record with a formal sign-off trail would ensure the limitations acknowledged here travel with the model rather than living only in a document.

13. Conclusion

The project delivered a complete machine learning system rather than a model: data arrives in an immutable lake and is cataloged for SQL exploration; features are versioned in a feature store with the label structurally isolated from the inference path; splits are chronological and their integrity is asserted by test; a candidate model must beat a measured floor before it can be registered and must be approved by a person before it can be deployed; it serves two ways from one artefact and the two paths are checked against each other; and monitors watch both the traffic and the model, with the label-free signal arriving first because it is the only one that arrives in time to matter.

What distinguishes the result from a modelling exercise is that most of the engineering went into the parts of the system that exist to catch mistakes: the leakage test, the quality gate and its failure branch, the parity check, the monitor thresholds, and the dry run. Each of those was written because a specific failure would otherwise have been invisible, and several of them caught exactly that failure during development. The two most instructive moments in the project — a monitor firing against its own baseline, and a perfectly calibrated predictor being charged with calibration error — were both produced by checks we had built to test something else.

The system's limitations are stated as plainly as its capabilities, because an operator needs both to use it responsibly. It runs on a sample rather than at full scale, its retraining trigger is designed rather than wired, and a fairness audit on protected attributes is impossible on hashed features and is therefore documented as out of scope rather than reported as passed. The production pipeline is registered and validated in SageMaker Studio, and its first quality verdict — a rejection on calibration — is evidence that the gate does what it says. On real data the same code produces a model with log loss 0.476 against a base rate of 0.572, AUC 0.772 and calibration error 0.005, which the gate accepts, while it rejects a reweighted variant with the same AUC: the gate separates two models that a ranking metric would call equal. Within those bounds, the system is reproducible, tested, monitored and gated — which is what the course set out to teach and what this project set out to demonstrate.

References

Criteo Labs. Display Advertising Challenge. Kaggle. https://www.kaggle.com/c/criteo-display-ad-challenge

Amazon Web Services. Amazon SageMaker Developer Guide — Feature Store, Model Monitor, Pipelines, Model Registry, Clarify, Debugger. https://docs.aws.amazon.com/sagemaker/

Chen, T. and Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining.

He, X. et al. (2014). Practical Lessons from Predicting Clicks on Ads at Facebook. Proceedings of the Eighth International Workshop on Data Mining for Online Advertising.

Guo, H. et al. (2017). DeepFM: A Factorization-Machine based Neural Network for CTR Prediction. Proceedings of the 26th International Joint Conference on Artificial Intelligence.

Mitchell, M. et al. (2019). Model Cards for Model Reporting. Proceedings of the Conference on Fairness, Accountability, and Transparency.

Ali, M. et al. (2019). Discrimination through Optimization: How Facebook's Ad Delivery Can Lead to Biased Outcomes. Proceedings of the ACM on Human-Computer Interaction.

Sculley, D. et al. (2015). Hidden Technical Debt in Machine Learning Systems. Advances in Neural Information Processing Systems 28.

Appendix A — Repository Map

Path	Contents
notebooks/01–11	The system in run order, from data lake to the production pipeline
src/criteo_ctr/	config, io_utils, athena, features, datasets, benchmark, evaluation, monitoring, deployment, pipeline, acquire — imported by notebooks, pipeline and tests alike
scripts/	preprocess.py and evaluate.py (pipeline steps), record_preprocessor.py (Model Monitor hook)
tests/	115 unit and integration tests plus the synthetic data generator
tools/	local_dry_run.py, make_architecture_diagram.py
docs/	architecture.md, data.md, model_card.md, runbook.md, operation_validation.md, sagemaker_studio_setup.md
reports/figures/	Generated figures, including the as-built architecture diagram
artifacts/	Run artefacts: manifests, encoder, evaluation payloads, pipeline definition
.github/workflows/	CI: tests, pipeline contract, dry run, notebook parse check, lint

Appendix B — Reproduction

Upload dac.tar.gz to s3://<bucket>/criteo-ctr/landing/. In SageMaker Studio, create a JupyterLab space with at least 30 GB of storage and add the SDK v2 kernel described in docs/sagemaker_studio_setup.md; on a notebook instance, use conda_python3 with a volume of at least 50 GB. Notebooks are executed in numerical order and each is idempotent. Without AWS, the test suite and the end-to-end dry run both execute locally after installing requirements.txt, and together they exercise every code path in the system apart from the AWS service calls themselves. To reproduce Section 6.3, place Criteo_1M_with_nans.csv in data/ and run python tools/run_1m_sample.py; it prints the split manifest, the metric table and the gate verdict, and writes the figures.

Appendix C — Monitoring Thresholds

Signal	Threshold	Rationale
Log loss, live versus approved baseline	+5% relative	Tolerates ordinary window-to-window variation while catching sustained degradation

ROC-AUC, live versus baseline	−0.02 absolute	Ranking loss of this size is operationally meaningful at bidding volume
Expected calibration error	0.01 absolute ceiling (gate: 0.02)	Calibration decays before ranking and directly mis-prices bids
Rare-bucket share per categorical column	+10 percentage points versus training	Relative because high-cardinality columns are naturally high in training; an absolute ceiling fires on healthy traffic
Population stability index per numeric feature	0.10 warn / 0.25 alarm	Conventional PSI bands
Mean predicted CTR	±20% relative	Detects distribution shift before labels arrive
Endpoint p99 model latency	100,000 microseconds	CloudWatch reports SageMaker latency in microseconds; a threshold written as if it were milliseconds never fires
Endpoint 5xx errors	≥1 in five minutes	Every server error is an unscored bid request
Host CPU and memory	80% sustained over two periods	Scale out before saturation

Appendix D — Deliverables

Deliverable	**Artefact**
1 — ML System Design Document	Final-Project_Group-4_Deliverable-1 (.docx / .pdf)
2 — ML System Operation Validation	Recorded demonstration, with the script and outline in Final-Project_Group-4_Deliverable-2
3 — Codebase	GitHub repository: https://github.com/sentinel1282/Group--4_MLOPs---Criteo_Advertisement_CTR_Prediction-
Supporting	This project report; the presentation deck; the executed production-pipeline notebook; the weekly team tracker for Modules 2–6